

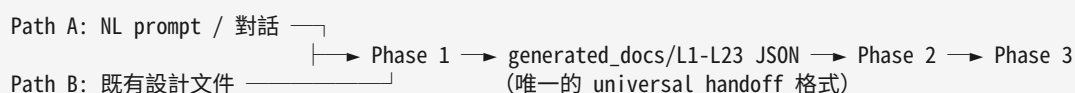
Vibe-IC 標準流程 v2.2.0 (繁體中文版)

Plugin 版本： 0.2.2 · **取代：** CANONICAL_FLOW_v2.0.0.md、PHASE_VS_STAGE_VS_INDUSTRY_TAXONOMY.md、docs/tutorials/33_step_flow_overview.md、docs/design/STANDARD_FLOW.md（皆停在 33-step / 54-entity 模型，早於 81-protocol Phase-1 dispatch、Phase-2 scaffold bridge、與 doc→GDS LVS 簽核鏈）。**英文正本：** CANONICAL_FLOW_v2.2.0.md（本檔為其繁中對照；如有歧義以英文正本為準）。

真實來源 = runner，而非本文件。 下列每一步都引用其 `*_one_shot_runner.py` 內的實際 step 標記。runner 一旦變更，請從標記重新產生本文件（每個 runner 跑 `grep -noE '[0-9]+[a-z0-9]*[0-9]+\|'`），不要手動遷移。

0. 兩個入口 + 編排器 (orchestrator)

programs/vibe ic one shot runner.py 是最上層編排器。兩個入口、一個交接格式：



編排順序 (vibe_ic_one_shot_runner.py) : **Phase 1** → **Phase 2 (=2a+2b)** → **Analog** (在 Phase 2 之後跑, 使 **L5_ADI_SPEC** 已填; 不阻斷) → **Phase 3**。FAIL 閘控: Phase 1 FAIL 在 Phase 2 前停; Phase 2 FAIL 在 Phase 3 前停; Phase 3 FAIL → 最終判決 FAIL 但仍輸出報告。

Phase	轉換	Runner	閘門
P0 預檢	環境 / PDK / 工具可用性	mcp_server_health_check 、 eda_doctor	工具可達
Phase 1	任何輸入 → L1-L23 JSON (+ 人讀 MD)	phase1_doc_one_shot_runner.py	24 份 L-doc + completeness/ parity
Phase 2	L1-L23 → 驗證過的 RTL → (gate netlist / FPGA SOF)	phase2_one_shot_runner.py	lint/synth/conformance/TB + final_audit
Analog A1-A8	L1/L5 → 定尺寸區塊 → hardmacro (與 P2 並行)	analog_one_shot_runner.py	各區塊 DRC/LVS + corners
Mixed-signal	數位 + 類比共模擬	skill mixed-signal-cosim (無專 屬 runner)	M1-M4 co-sim
Phase 3	netlist → synth → PnR → GDS → DRC → LVS	phase3_one_shot_runner.py	DRC 0 + LVS + STA + tapeout checklist

1. Phase 1 – 15 步 (phase1 doc one shot runner.py 的 [N/15] 標記)

標記	步驟	輸出
[1/15]	從 input/docs/ 萃取文字 (PDF/DOC/XLSX/...)	input_doc/ (對 O(n ²) 掃描每份上限 2 MB , v0.1.91)
[2/15]	L1_DATASHEET	generated_docs/L1_DATASHEET.json
[3/15]	L2_FRS	L2
[4/15]	L3_CMD_PROTOCOL	L3
[5/15]	L4_REGMAP	L4
[6/15]	L5_ADI_SPEC	L5
[7/15]	L6_CONTROL_LOGIC	L6
[8/15]	L7_TEST_DEBUG	L7

標記	步驟	輸出
[9/15]	L8_RTL_CONSTANTS	L8_RTL_CONSTANTS
[10/15]	L9_INTEGRATION_SPEC	L9
[11/15]	L10_TEST_CASES	L10
[12/15]	L11_OTP_CONTENT	L11
[13/15]	L12_BEHAVIORAL_SEQUENCES	L12
[14/15]	L13_LAB_CALIBRATION	L13
[14b/15]	L8_TIMING_WAVEFORM (+ 14b2 寬度、14b3 編碼表、14b7 通用常數、14b4 L6 FSM、14b5 L12 序列)	L8_TIMING_WAVEFORM + overlays
[14c/15]	L14-L18 協定規格萃取 (+ 14c0 L9、14c1 L1 meta、14c1b L17 handshake、14c2 L3 mirror、14c3 L17/L18/L8_TIMING/L9 批次 synth、14c4 通用 doc facts、14c5 殘餘清理)	L14-L18
[14d/15]	L19-L23 skeleton 產出	L19-L23
[14e/15]	serial_peripheral_protocol 類別 synth (R53/R54/R55) — 協定 detector→synth dispatch	各協定 L-doc overlay
[14e2/15]	bus_interconnect_protocol Tier-2 synth (TileLink/Wishbone/Avalon/OCP/AXI-Stream)	bus 協定 overlay
[14e3/15]	通用 packet/PDU L10↔L3 opcode 一致性掃描	L10 清理
[15/15]	Coverage / parity 報告	reports/phase1/

[14e/15] 這個區塊就是 **81 個協定類別的引擎**（本 session，v0.1.84→v0.2.2）。每個協定提供一個 content-only 的 `is_<proto>(blob) detector + <proto>.protocol_synth.py` overlay，依 `ic_class` 分派。家族：序列（SPI/I2C/I3C/UART/1-Wire/JTAG/SWD/QSPI/SMBus/MIPI-SPMI-RFFE）、車用（CAN/CAN-FD/LIN/FlexRay/SENT/PSI5/Modbus/RS-485/CANopen）、記憶體（DDR3/4/5/LPDDR5/HBM3/ GDDR6/ONFI/eMMC/SD-MMC/UFS/NVMe/HyperBus）、顯示（HDMI/MIPI/DSI/CSI-2/DisplayPort/eDP）、PCIe 家族（PCIe/Gen5/CXL/NVLink/UCIe）、USB（2.0/USB4）、網路（Ethernet/800G/HDLC/SpaceWire/AFDX/ Auto-Eth/InfiniBand/FibreChannel/PROFIBUS/PROFINET/IO-Link/EtherCAT）、無線（BLE/NFC/Zigbee/LoRa）、音訊（I2S/SoundWire/S-PDIF/A2B）、資料轉換（JESD204）、除錯（CoreSight）、時間同步（PTP）、航太（MIL-STD-1553/ARINC429）、bus（AHB-APB/AXI-ACE-CHI/TileLink/Wishbone/Avalon/OCP/AXI-Stream）、安全（TPM）。**守衛**：每個 module-level detector 由 `tests/test_protocol_detector_no_misfire.py` 自動納管（不誤觸外部 benchmark；衍生兄弟允許清單在 `protocol_detector_lib.DERIVED_SIBLING_CROSS_FIRES`）。

2. Phase 2 — RTL 撰寫 + 驗證（`phase2_one_shot_runner.py`，`step_*`）

順序	步驟（ <code>def step_*</code> ）	說明
1	<code>step_phase1</code>	必要時重跑/匯入 Phase 1
2	<code>step_rig_topology_skeleton</code>	拓樸 scaffold
3	<code>step_rtl_gen</code>	對 <code>rtl_gen=null</code> 的 <code>ic_class</code> 會 WAIVE → AI 在 pipeline 內扮演 <code>spec-to-rtl</code> 角色（ <code>digital_arithmetic_primitive / digital_cmd_driven / serial_peripheral_protocol / bus_interconnect_protocol / processor_cpu / unknown</code> ）。 <code>phase2_scaffold_gen.py</code> 決定性地產出 <code>top/regs/fsm/tb/soc_wrap/cocotb scaffold</code> 。
4	<code>step_full_stack_tb_gen</code>	自檢 TB 產生

順序	步驟 (def step_*)	說明
5	step_reference_tb	reference-TB 一致性 (eco_loop 最多 3 次重試)
6	step_yosys_synth	gate-level 合成
7	step_qsf_gen / step_sdc_gen	FPGA 專案 + 約束
8	step_otp_image_check	OTP image (若適用)
9	step_fpga_compile	Quartus/yosys FPGA build → .sof
10	step_fpga_burn	燒錄板子 (硬體路徑)
11	step_usb_hid_tester_verify	host 端協定測試器驗收
12	step_emit_phase2_manifests	phase2 manifests
13	step_final_audit	彙整審查關門

環繞 AI 撰寫步驟的決定性 gate：rtl_hygiene_lint --fix、spec_conformance_check、chip_top_gate_wrapper_gen、MCP eda_lint/eda_synth/eda_cocotb。

3. Analog A1-A8 (analog_one_shot_runner.py，與 Phase 2 並行)

步驟	名稱	輸出
A1	spec_extract	analog/<block>/A1_spec.json
A2	topology_select	A2_topology.json
A3	netlist_gen	<block>.sp
A4	corner_sweep	A4_corners.json
A5	layout (Magic)	A5_layout.json (需 DRC-clean + LVS-match flag)
A6	post_layout_resim	A6_postsim.json
A7	hardmacro_gen	{.lef,.lib,.gds,.v} → 餵回 Phase 3
A8	hw_verify (HIL)	A8_hw_verify.json

(舊文件寫「A1-A9」；runner 實際是 **A1-A8**。Mixed-signal **M1-M4** 為 skill 層級 mixed-signal-cosim，無專屬 *_one_shot_runner.py。)

4. Phase 3 — 實體設計 + 簽核 (phase3_one_shot_runner.py，step_*)

步驟	名稱	工具 (開源替代)
1	step_synth	yosys (sky130/gf180) — + tie-cell pass (見 §5)
2	step_pnr	OpenROAD (floorplan → PDN → place → CTS → route)
3	step_gds	KLayout streamout (def2gds；OpenROAD 已不再 stream GDS)
4	step_drc	KLayout sky130 deck
5	step_lvs	netgen / yosys_equiv — §5 的簽核鏈
6	step_canonicalize_artefacts	正規化輸出

Tapeout 關門：tapeout-checklist (DRC/LVS/STA/IR/EM/antenna/ERC/LEC/DFT) + flow_compliance_check。

5. LVS 簽核鏈 — 本 era 全新 (v0.1.96 → v0.2.2)

這是舊文件**完全缺少**的部分。step_lvs 已不再是單一 yosys_equiv 呼叫，而是一條分層鏈，每一層都被 7 個 doc→GDS pilot 打磨硬化：

1. **Structural LEC (預設)** — eda_lvs mode=yosys_equiv (equiv_simple+equiv_induct)。殘留的「unproven」cell = yosys 對 PDK primitive 的 SAT-model 落差 (Category-D 工具限制，**非 mismatch**)。沒有任何 yosys flag 能對所有 cell 關掉它。
2. **Device-level 覆蓋** — 要覆蓋 SAT 落差：eda_extraction (magic ext2spice) + eda_lvs mode=netgen + lvs_netgen_setup_emit.py (電源網路 globalization)。比對的是電晶體 → 沒有 SAT-model 概念。可達 device-class-exact (HDL 20937=20937；sha256 12148=12148)。
3. **Powered-netlist 收尾** — OpenROAD write_verilog -include_pwr_gnd (在 global_connect 之後) 讓 schematic 側帶真實 VPWR/VGND/VPB/VNB → 消除 tie-cell 斷接節點殘留。
4. **頂層 port label** — Route A (治本)：magic_port_extract_emit.py (export PDK + port makeall)。Route B (備援)：lvs_def_port_seed.py (解析 DEF PINS)。(實證：Route A 才是必要的；B 只是稽核提示。)
5. **Sign-off guard (強制)** — lvs_signoff_guard.py：當「match」是針對 **portless** 的 extracted top .subckt (vacuous / 靜默假陽性 條件) 時直接 RAISE。在信任任何 LVS match 前先跑它。

Synth tie-cell 預步 (§ 4 step 1)：setundef -zero; hilomap -hicell conb_1 HI -locell conb_1 L0; splitnets; clean (不用 opt_clean——它會刪掉 tie cell)。沒有它，常數網路會在 TritonRoute 觸發 DRT-0305。bare MCP eda_synth 路徑缺這步 (backlog ORGANIC-20260531-mcp-eda-synth-missing-hilomap-tiecells)；phase3_one_shot_runner 會自動做。前向驗證 SENT→QSPI→HDL→SpaceWire。

5b. Phase-3 簽核檢查 — 缺口狀態

step_drc/step_lvs 是頭條閘門，但完整簽核還會跑更多檢查。下表是**簽核檢查與其狀態**（不是每個 實體設計步驟——placement / CTS / routing / 輸出 / ECO 見 § 4 與 § 6 pilot，皆 PASS）。**下列沒有一個是電路設計錯誤**——都是腳本順序 / 級聯 / 環境 / 報告格式 問題。嚴重度與根因（已稽核）：

編號注意（這就是「step 23」看似不見的原因）：有兩套編號。下表用**簽核稽核**那套 (SPEF 22 / STA 23 / IR 24 / EM 25 / Antenna 26 / SI 27 / DRC-LVS-ERC 30 / fill 33)；repo 的 33_step_flow_overview.md 對同樣的檢查用另一套 (SPEF 20 / STA 21 / IR 22 / EM 23 / Antenna 24 / SI 25 / PV 27)。待 flow_doc_emit.py (§ 8) 落地時統一成一套。**step 23 = Post-route STA**，下表列為 PASS——先前缺它，只是因為以缺口為主的草稿略過了通過的檢查。

步	檢查	一句話	開源狀態	嚴重度
22	SPEF (OpenRCX)	逐線 R/C「寄生」萃取 — 餵給 STA/IR/EM/SI	extract.tcl 必須先 global_route + set_wire_rc，否則無 SPEF	◆ 中 (腳本順序)
23	Post-route STA (MMMC)	多角落靜態時序簽核	SPEF 一有就跑；pilot 回報 setup slack +X ns MET (3 角落)	● 無 (通過)
24	IR drop (PSM)	開關電流下的電源網路電壓降	級聯缺失：SPEF(22) 一有就解鎖	◆ 中 (級聯)
25	EM	電遷移 — 電流密度長期侵蝕金屬	級聯缺失：需 SPEF(22)	◆ 中 (級聯)
27	SI	串擾 — 一條線的跳變耦合到鄰線	級聯缺失：需 SPEF(22)	◆ 中 (級聯)
26	Antenna	長導線累積電漿電荷 → 擊穿閘極	OpenROAD 繞線器已內建檢查；報告不在 audit 路徑	● 低 (已做，報告路徑)
30	DRC / LVS / ERC	製程規則 / 佈局對電路圖 / 電氣規則	sky130 PDK 只附 Calibre deck；開源需接 KLayout/Magic deck (本 era 已加)	● 高 (環境 / deck)

步	檢查	一句話	開源狀態	嚴重度
			§ 5 的 device-level netgen LVS 鏈)	
33	Metal fill	CMP 密度均勻用的假金屬	runner 缺 fill 階段 → 無 filled.def	◆ 中 (缺階段)
18	Spare cells	預留的 tied-off ECO 備用單元	30 個 spare 已正確放置； spare_cells.json 缺 rows[] 欄位 → audit 讀不到	● 低 (報告 schema)
5	Formal	SAT/model-checking 證明 (相對於抽樣模擬)	altsyncram primitive 無 formal model → INFORMATIONAL waiver (功能由 post-layout sim 步 28 覆蓋)	● 無 (已 waive)

Doctrine：每項都像 LVS 鏈 (§ 5) 一樣處理——把「設計正確性」訊號 (這裡：clean) 與「工具/腳本/報告」訊號 (這裡：可行的缺口) 分開。可行的修法 (SPEF extract.tcl 順序；它解鎖的 IR/EM/SI 級聯；開源 DRC/LVS deck；metal-fill 階段；spare_cells.json schema 欄位) 追蹤於 `ORGANIC-20260531-phase3-signoff-chain-open-source-gaps`。

6. doc→GDS pilot 證據 (7 個 pilot，真實 sky130A GDS)

Pilot	原型	ic_class	LVS 停點
i2s	streaming-rx	digital_cmd_driven	device-level exact 4499=4499 (3 SAT-unproven → 0；0 tie cell，port-label floor)
ahb_apb	bus-bridge	bus_interconnect	—
ufs	storage-framer	serial_peripheral	—
sent	sensor-decoder	digital_arithmetic_primitive	structural 全證 1388/1388
qspi	command-controller	serial_peripheral	structural 全證 1434/1434
hdlc	packet-framer	digital_cmd_driven	device-level exact 20937=20937 (SAT 落差 → 0)
spacewire	link credit-flow-control	digital_arithmetic_primitive	device-level exact 6676=6676 / powered 6164=6164 (99 SAT-unproven → 0；port-label floor)

三個做了 device-level 的 pilot (i2s + hdlc + spacewire) 現在共用同一個停點 — § 5 LVS 鏈端到端：structural-LEC SAT 殘留 → device-level netgen (覆蓋到 device-class-exact，每個 SAT-unproven cell → 0) → powered-netlist (消除 tie-cell 電源-pin 節點；i2s 無 tie cell 故 N/A) → 殘留 = Category-D port-label floor (port makeall / sign-off LVS)，lvs_signoff_guard 正確拒絕 vacuous portless match。已無任何 pilot 停在 structural-SAT 落差。

7. 專案資料夾結構 (與 v2.0.0 相同)

```
<project>/
input/{docs,phase1_prompt.md}      input_doc/          (Phase 1 輸入/萃取)
phase1/{generated_docs,human_docs,claude_extracted}/L*.json|md
phase2/stage1/{rtl,scaffold,fpga}/  phase2/stage2/synth/
analog/<block>/{A1_spec.json,...,<block>.{sp,lef,lib,gds,v}}
phase3/stage{1..4}/{synth,pnr,gds,drc,lvs}/  phase3/stage5_manufacturing/
reports/{phase1,phase2,phase3,orchestrator}/
```

8. 保持更新

本文件是**衍生的**，非權威來源。runner 變更後重新產生：對 `phase1_doc_one_shot_runner.py` 跑 `grep -noE '\[[0-9]+[a-z0-9]*/[0-9]+\][^"]*'`，對 `phase{2,3}_one_shot_runner.py` 跑 `grep -noE 'def step_[a-z0-9_]+'`，analog 取 `analog_one_shot_runner.py` 標頭的 A1-A8 區塊。未來強化（backlog 候選）：做一支 `programs/flow_doc_emit.py`，像 `INDEX.md` 一樣從 runner 標記決定性地 產出本表。